UNIVERSITY OF PISA

# Lesson 3

Silvia Giulia Galfrè

# Finding Protein Motifs

- Protein motifs are often associated with a specific function
- **Prosite** database (https://prosite.expasy.org/) to search motifs
- Motifs are represented in different formats; some syntax rules are:
  - Amino acid represented as one letter
  - X represent any amino acid in that position
  - A list of amino acids in [ ] represent all possible amino acids in a given position
  - A number in () means occurrences of those amino acids
  - A pair of numbers (*n,m*) indicate number or occurrences between *n* and *m*
  - - separate several positions
- Example
  - The zinc finger RING-type signature is "C-x-H-x-[LIVMFY]-C-x(2)-C-[LIVMYA] "

Let's code

# Homework

- In prosite check which is the pattern for ATP-dependent DNA ligase (what is it?).
- Find two protein Fasta files: one corresponding to a ATP-dependent DNA ligase and another that is not.
- Check if the function written in class recognize correctly the two proteins

approximate pattern matching

# PAIRWISE SEQUENCE ALIGNMENT

# Comparing strings: approximate pattern matching

- Consider that we have a fixed pattern, but we do not need to find an exact match
- Similar sequences are those share  same symbols in the same order… it seams trivial but it is not!
- Source of variability: mutations… change in one or more nucleotides but also insertion and deletions…
- ***Sequence alignment:*** a computational procedure for seeking shared individual characters in both sequences, keeping the order.

# Comparing strings

Two broad classes of sequence alignment:

- Global: we want to align the complete sequences

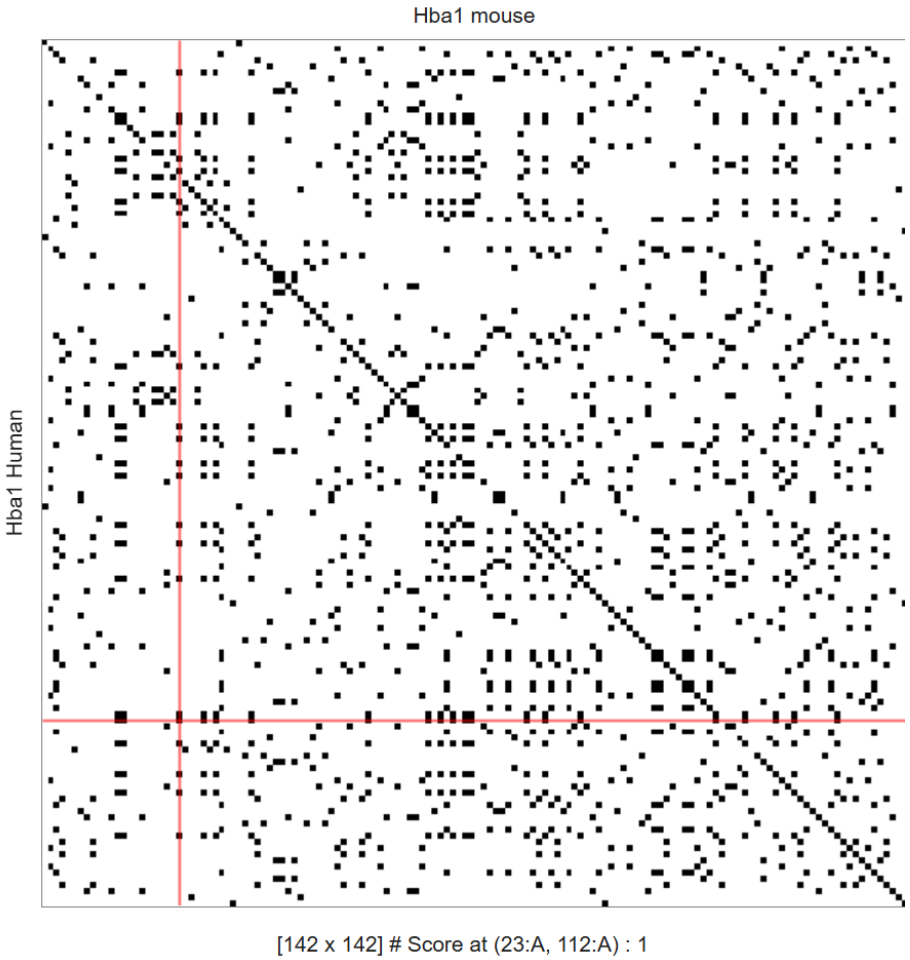
```
-CAGTGCATG-ACATA
 ||| || |  ||| |
TCAG-GC-TCTACAGA
```

- Local : we want to find a good alignment of adjacent parts of both sequences

```
-CAGTGCATGTACAGA
        ||||||||
 TTCCTCTGTACAGA
```

# Comparing strings

- Sometimes it can be useful to compare sequences visually and identify local similarities (not only global)
- Dot plot
  - for each position *i* in the first string and each position *j* in the second, draw a point if the following **w** characters are identical




https://dotlet.vital-it.ch/

# Comparing strings

- We need a means to measure how good a partial match is
  - Distance or similarity measures
    - Formally - metrics  or distance functions
      - Distance to itself is always 0, **d(a,a)=0**
      - Distance from **a** to **b** is equal to distance from **b** to **a**, **d(a,b)=d(b,a)**
      - The distance between two distinct points is always positive, **a!=b => d(a,b)>0**
      - Having **a,b,c, d(a,c)<=d(a,b)+d(b,c)**  (triangular inequality)
  - Distances between strings
    - Hamming distance
    - Edit distance

# Comparing strings

- Hamming distance
  - For strings of same size, measures the number of positions on which the characters are different

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **S1** | A | A | C | G | C | G | T | A | T | T | G | C | C | G | G |
| **S2** | A | A | C | G | A | G | G | A | T | A | C | C | G | G | G |

$\mathbf{d_{hamming}(S1,S2)=5}$

# Edit distance

- Can be used for strings of any length and counts the minimum number of single-character insertions, deletions, or substitutions required to transform one string into the other
- **d(S1, S2)**: The number (or total cost) of *edit operations* that are required to transform **S1** into **S2**
- Each edit operation is assigned a cost
  - Edit operations
  - **Substitute** one character with another - works for **strings of same length**

# Alignment as optimization problem

- Align two strings -> several solutions
- We need to find the "best"

$$\binom{2n}{n} = \frac{(2n)!}{n!^2}$$

**Inputs:**

- Two sequences,
- **An objective function**

**Outputs:**

The optimal paring

# An objective function

- It has the purpose of  evaluating all possible feasible solutions
- It assign to each solution a score

# The simplest objective function

- Additive: counting the number of equal symbols

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **S1** | A | A | C | G | C | G | T | A | T | T | G | C | C | G | G |
| **S2** | A | A | C | G | A | G | G | A | T | A | C | C | G | G | G |

10

# The simplest objective function

| | A | A | C | G | C | G | T | A | T | T | G | C | C | G | G |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| A | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| A | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| C | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 0 |
| G | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | 1 |
| A | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| G | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | 1 |
| G | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | 1 |
| A | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| T | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 0 |
| A | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| C | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 0 |
| C | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 0 |
| G | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | 1 |
| G | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | 1 |
| G | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | 1 |

With gaps
11

| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| A | A | C | G | C | G | T | A | T | T | G | C | C | G | G | |
| A | A | C | G | A | G | G | A | T | A | | C | C | G | G | G |

# More complex objective functions

- Addictive on each corresponding symbol
- Penalty for each mismatch or gap

# Common for DNA/RNA

- Addictive objective function
- But different scores:
  - Matches +1 or +2
  - Mismatches or gaps -3 or -2

Not used for proteins

# Different substitution costs

- Since the effect of changing an amino acid depend also from chemistry of the two aa, the costs are different
- Similar aa will have lower cost
- Very different aa will have higher costs

How to define these costs?

# BLOSUM

- BLOcks of amino acids Substitution Matrix
- The **BLOSUM** family was introduced by **Henikoff & Henikoff (1992)** to quantify how likely one amino acid is to be substituted by another in *real* protein evolution.
- Alignments of evolutionarily related protein sequences in highly conserved regions (blocks), where the alignment is trusted to be correct – the "good alignment"

# BLOSUM

- Based on the probability of finding a given pair of aa aligned in a "good alignment"
- So is based on the frequencies assuming independence in the occurrence of each aa

$$s(a,b) = round(2 * \log_2 \frac{\mathrm{P_{(a,b)}}}{\mathrm{P_a P_b}})$$

- Example: 1000 aa pairs in total, SL (serine - lysine) occurs 9 times, frequency of S 15%, frequency of L 10%:
  - $Score(S,L) \;\; = \;\; round(2 * \log_2 \frac{9/1000}{0.1*0.15})$

# BLOSUM62

BLOSUM50 and BLOSUM62 are the most used versions

- Built using alignment with similarity level of over 62%
- Hydrophobic
- Special chases
- Polar, uncharged
- Polar +
- Polar -

| | C | S | T | A | G | P | D | E | Q | N | H | R | K | M | I | L | V | W | Y | F | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| C | 9 | | | | | | | | | | | | | | | | | | | | C |
| S | -1 | 4 | | | | | | | | | | | | | | | | | | | S |
| T | -1 | 1 | 5 | | | | | | | | | | | | | | | | | | T |
| A | 0 | 1 | 0 | 4 | | | | | | | | | | | | | | | | | A |
| G | -3 | 0 | -2 | 0 | 6 | | | | | | | | | | | | | | | | G |
| P | -3 | -1 | -1 | -1 | -2 | 7 | | | | | | | | | | | | | | | P |
| D | -3 | 0 | -1 | -2 | -1 | -1 | 6 | | | | | | | | | | | | | | D |
| E | -4 | 0 | -1 | -1 | -2 | -1 | 2 | 5 | | | | | | | | | | | | | E |
| Q | -3 | 0 | -1 | -1 | -2 | -1 | 0 | 2 | 5 | | | | | | | | | | | | Q |
| N | -3 | 1 | 0 | -2 | 0 | -2 | 1 | 0 | 0 | 6 | | | | | | | | | | | N |
| H | -3 | -1 | -2 | -2 | -2 | -2 | -1 | 0 | 0 | 1 | 8 | | | | | | | | | | H |
| R | -3 | -1 | -1 | -1 | -2 | -2 | -2 | 0 | 1 | 0 | 0 | 5 | | | | | | | | | R |
| K | -3 | 0 | -1 | -1 | -2 | -1 | -1 | 1 | 1 | 0 | -1 | 2 | 5 | | | | | | | | K |
| M | -1 | -1 | -1 | -1 | -3 | -2 | -3 | -2 | 0 | -2 | -2 | -1 | -1 | 5 | | | | | | | M |
| I | -1 | -2 | -1 | -1 | -4 | -3 | -3 | -3 | -3 | -3 | -3 | -3 | -3 | 1 | 4 | | | | | | I |
| L | -1 | -2 | -1 | -1 | -4 | -3 | -4 | -3 | -2 | -3 | -3 | -2 | -2 | 2 | 2 | 4 | | | | | L |
| V | -1 | -2 | 0 | 0 | -3 | -2 | -3 | -2 | -2 | -3 | -3 | -3 | -2 | 1 | 3 | 1 | 4 | | | | V |
| W | -2 | -3 | -2 | -3 | -2 | -4 | -4 | -3 | -2 | -4 | -2 | -3 | -3 | -1 | -3 | -2 | -3 | 11 | | | W |
| Y | -2 | -2 | -2 | -2 | -3 | -3 | -3 | -2 | -1 | -2 | 2 | -2 | -2 | -1 | -1 | -1 | -1 | 2 | 7 | | Y |
| F | -2 | -2 | -2 | -2 | -3 | -4 | -3 | -3 | -3 | -3 | -1 | -3 | -3 | 0 | 0 | 0 | -1 | 1 | 3 | 6 | F |
| | C | S | T | A | G | P | D | E | Q | N | H | R | K | M | I | L | V | W | Y | F | |

# PAM

- Point Accepted Mutations
- Derived from closely related protein families with known evolutionary distance.
- Since it is a distance, lower is the number closer are the aa

# BLOSUM50

- Exercise:
  - Score the HBA-HBB alignment using BLOSUM50
  - S= s(G,G)+s(S,N)+s(A,P)+.... =
  - =8+1-1+.... =130

```
HBA_HUMAN   GSAQVKGHGKKVADALTNAVAHVDDMPNALSALSDLHAHKL
            G+  +VK+HGKKV   A+++++AH+D++  +++++LS+LH   KL
HBB_HUMAN   GNPKVKAHGKKVLGAFSDGLAHLDNLKGTFATLSELHCDKL
```

| | A | R | N | D | C | Q | E | G | H | I | L | K | M | F | P | S | T | W | Y | V |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| A | **5** | −2 | −1 | −2 | −1 | −1 | −1 | 0 | −2 | −1 | −2 | −1 | −1 | −3 | −1 | 1 | 0 | −3 | −2 | 0 |
| R | −2 | **7** | −1 | −2 | −4 | 1 | 0 | −3 | 0 | −4 | −3 | 3 | −2 | −3 | −3 | −1 | −1 | −3 | −1 | −3 |
| N | −1 | −1 | **7** | 2 | −2 | 0 | 0 | 0 | 1 | −3 | −4 | 0 | −2 | −4 | −2 | 1 | 0 | −4 | −2 | −3 |
| D | −2 | −2 | 2 | **8** | −4 | 0 | 2 | −1 | −1 | −4 | −4 | −1 | −4 | −5 | −1 | 0 | −1 | −5 | −3 | −4 |
| C | −1 | −4 | −2 | −4 | **13** | −3 | −3 | −3 | −3 | −2 | −2 | −3 | −2 | −2 | −4 | −1 | −1 | −5 | −3 | −1 |
| Q | −1 | 1 | 0 | 0 | −3 | **7** | 2 | −2 | 1 | −3 | −2 | 2 | 0 | −4 | −1 | 0 | −1 | −1 | −1 | −3 |
| E | −1 | 0 | 0 | 2 | −3 | 2 | **6** | −3 | 0 | −4 | −3 | 1 | −2 | −3 | −1 | −1 | −1 | −3 | −2 | −3 |
| G | 0 | −3 | 0 | −1 | −3 | −2 | −3 | **8** | −2 | −4 | −4 | −2 | −3 | −4 | −2 | 0 | −2 | −3 | −3 | −4 |
| H | −2 | 0 | 1 | −1 | −3 | 1 | 0 | −2 | **10** | −4 | −3 | 0 | −1 | −1 | −2 | −1 | −2 | −3 | 2 | −4 |
| I | −1 | −4 | −3 | −4 | −2 | −3 | −4 | −4 | −4 | **5** | 2 | −3 | 2 | 0 | −3 | −3 | −1 | −3 | −1 | 4 |
| L | −2 | −3 | −4 | −4 | −2 | −2 | −3 | −4 | −3 | 2 | **5** | −3 | 3 | 1 | −4 | −3 | −1 | −2 | −1 | 1 |
| K | −1 | 3 | 0 | −1 | −3 | 2 | 1 | −2 | 0 | −3 | −3 | **6** | −2 | −4 | −1 | 0 | −1 | −3 | −2 | −3 |
| M | −1 | −2 | −2 | −4 | −2 | 0 | −2 | −3 | −1 | 2 | 3 | −2 | **7** | 0 | −3 | −2 | −1 | −1 | 0 | 1 |
| F | −3 | −3 | −4 | −5 | −2 | −4 | −3 | −4 | −1 | 0 | 1 | −4 | 0 | **8** | −4 | −3 | −2 | 1 | 4 | −1 |
| P | −1 | −3 | −2 | −1 | −4 | −1 | −1 | −2 | −2 | −3 | −4 | −1 | −3 | −4 | **10** | −1 | −1 | −4 | −3 | −3 |
| S | 1 | −1 | 1 | 0 | −1 | 0 | −1 | 0 | −1 | −3 | −3 | 0 | −2 | −3 | −1 | **5** | 2 | −4 | −2 | −2 |
| T | 0 | −1 | 0 | −1 | −1 | −1 | −1 | −2 | −2 | −1 | −1 | −1 | −1 | −2 | −1 | 2 | **5** | −3 | −2 | 0 |
| W | −3 | −3 | −4 | −5 | −5 | −1 | −3 | −3 | −3 | −3 | −2 | −3 | −1 | 1 | −4 | −4 | −3 | **15** | 2 | −3 |
| Y | −2 | −1 | −2 | −3 | −3 | −1 | −2 | −3 | 2 | −1 | −1 | −2 | 0 | 4 | −3 | −2 | −2 | 2 | **8** | −1 |
| V | 0 | −3 | −3 | −4 | −1 | −3 | −3 | −4 | −4 | 4 | 1 | −3 | 1 | −1 | −3 | −2 | 0 | −3 | −1 | **5** |

# Score (substitution) matrices in Biopython

- Module Bio.Align.substitution_matrices
- Function `load()`
  - Without parameters shows all available precomputed matrices
  - With the name as parameter, returns the corresponding matrix
- Function `read()`
  - Reads a matrix from a file handler.

# Score (substitution) matrices in Biopython

```
from Bio.Align import substitution_matrices as sm
print(sm.load())
```

```
['BENNER22', 'BENNER6', 'BENNER74', 'BLOSUM45', 'BLOSUM50', 'BLOSUM62',
'BLOSUM80', 'BLOSUM90', 'DAYHOFF', 'FENG', 'GENETIC', 'GONNET1992', 'HOXD
70', 'JOHNSON', 'JONES', 'LEVIN', 'MCLACHLAN', 'MDM78', 'NUC.4.4', 'PAM25
0', 'PAM30', 'PAM70', 'RAO', 'RISLER', 'SCHNEIDER', 'STR', 'TRANS']
```

# Score (substitution) matrices in Biopython

```
print(sm.load("BLOSUM50"))
```

```
#  Matrix made by matblas from blosum50.iij
#  * column uses minimum score
#  BLOSUM Clustered Scoring Matrix in 1/3 Bit Units
#  Blocks Database = /data/blocks_5.0/blocks.dat
#  Cluster Percentage: >= 50
#  Entropy =   0.4808, Expected =  -0.3573
     A    R    N    D    C    Q    E    G    H    I    L    K    M    F    P    S    T    W    Y    V    B    Z    X
*
A  5.0 -2.0 -1.0 -2.0 -1.0 -1.0 -1.0  0.0 -2.0 -1.0 -2.0 -1.0 -1.0 -3.0 -1.0  1.0  0.0 -3.0 -2.0  0.0 -2.0 -1.0 -1.0
-5.0
R -2.0  7.0 -1.0 -2.0 -4.0  1.0  0.0 -3.0  0.0 -4.0 -3.0  3.0 -2.0 -3.0 -3.0 -1.0 -1.0 -3.0 -1.0 -3.0 -1.0  0.0 -1.0
-5.0
N -1.0 -1.0  7.0  2.0 -2.0  0.0  0.0  0.0  1.0 -3.0 -4.0  0.0 -2.0 -4.0 -2.0  1.0  0.0 -4.0 -2.0 -3.0  4.0  0.0 -1.0
-5.0
D -2.0 -2.0  2.0  8.0 -4.0  0.0  2.0 -1.0 -1.0 -4.0 -4.0 -1.0 -4.0 -5.0 -1.0  0.0 -1.0 -5.0 -3.0 -4.0  5.0  1.0 -1.0
-5.0
C -1.0 -4.0 -2.0 -4.0 13.0 -3.0 -3.0 -3.0 -3.0 -2.0 -2.0 -3.0 -2.0 -2.0 -4.0 -1.0 -1.0 -5.0 -3.0 -1.0 -3.0 -3.0 -2.0
-5.0
Q -1.0  1.0  0.0  0.0 -3.0  7.0  2.0 -2.0  1.0 -3.0 -2.0  2.0  0.0 -4.0 -1.0  0.0 -1.0 -1.0 -1.0 -3.0  0.0  4.0 -1.0
-5.0
E -1.0  0.0  0.0  2.0 -3.0  2.0  6.0 -3.0  0.0 -4.0 -3.0  1.0 -2.0 -3.0 -1.0 -1.0 -1.0 -3.0 -2.0 -3.0  1.0  5.0 -1.0
-5.0
G  0.0 -3.0  0.0 -1.0 -3.0 -2.0 -3.0  8.0 -2.0 -4.0 -4.0 -2.0 -3.0 -4.0 -2.0  0.0 -2.0 -3.0 -3.0 -4.0 -1.0 -2.0 -2.0
-5.0
H -2.0  0.0  1.0 -1.0 -3.0  1.0  0.0 -2.0 10.0 -4.0 -3.0  0.0 -1.0 -1.0 -2.0 -1.0 -2.0 -3.0  2.0 -4.0  0.0  0.0 -1.0
-5.0
```

# Exercises

Write a Python function that scores a given pairwise alignment, represented as two Seq objects. The function takes as input the sequences, and a substitution matrix and returns the score. Apply to the first alignment on slide 4, using BLOSUM50 and PAM250 matrices.

➔GSAQVKGHGKKVADALTNAVAHVDDMPNALSALSDLHAHKL

➔GNPKVKAHGKKVLGAFSDGLAHLDNLKGTFATLSELHCDKL

Compare the PAM250 and BLOSUM50 matrices using a scatter plot: include one point for each amino acid pair

# Gaps

- Simplest: constant penalty
  - For proteins typically [-12,-7]
  - For DNA/RNA typically [-3,-2]
- More used *affine gap penalty*: it penalize more the starting of a gap but less the extension (typically -1, -2)

Code with BioPython

biopython

Python Tools for Computational Molecular Biology

Documentation
Download
Mailing lists
News
Biopython Contributors
Scriptcentral
Source Code

# Biopython

See also our News feed.

## Introduction

Biopython is a set of freely available tools for biological computation written in Python by an international team of developers.

It is a distributed collaborative effort to develop Python libraries and applications which address the needs of current and future work in bioinformatics. The source code is made available under the Biopython License, which is extremely liberal and compatible with almost every license in the world.

We are a member project of the Open Bioinformatics Foundation (OBF), who take care of our domain name and hosting for our mailing list etc. The OBF used to host our development repository, issue tracker and website but these are now on GitHub.

This page will help you download and install Biopython, and start using the libraries and tools.

# INTRO TO BIOPYTHON

# Biopython Transcription

```
      DNA coding strand (aka Crick strand, strand +1)
5'    ATGGCCATTGTAATGGGCCGCTGAAAGGGTGCCCGATAG    3'
      |||||||||||||||||||||||||||||||||||||||
3'    TACCGGTAACATTACCCGGCGACTTTCCCACGGGCTATC    5'
   DNA template strand (aka Watson strand, strand -1)
```

Transcription of this DNA sequence produces the following RNA sequence:

```
5'    AUGGCCAUUGUAAUGGGCCGCUGAAAGGGUGCCCGAUAG    3'
          Single-stranded messenger RNA
```

```
>>> from Bio.Seq import Seq
>>> coding_dna = Seq("ATGGCCATTGTAATGGGCCGCTGAAAGGGTGCCCGATAG")
>>> coding_dna
Seq('ATGGCCATTGTAATGGGCCGCTGAAAGGGTGCCCGATAG')
>>> template_dna = coding_dna.reverse_complement()
>>> template_dna
Seq('CTATCGGGCACCCTTTCAGCGGCCCATTACAATGGCCAT')
```

# Biopython Transcription

```
      DNA coding strand (aka Crick strand, strand +1)
5'    ATGGCCATTGTAATGGGCCGCTGAAAGGGTGCCCGATAG    3'
      |||||||||||||||||||||||||||||||||||||||
3'    TACCGGTAACATTACCCGGCGACTTTCCCACGGGCTATC    5'
   DNA template strand (aka Watson strand, strand -1)
```

Transcription of this DNA sequence produces the following RNA sequence:

```
5'    AUGGCCAUUGUAAUGGGCCGCUGAAAGGGUGCCCGAUAG    3'
          Single-stranded messenger RNA
```

```
>>> coding_dna
Seq('ATGGCCATTGTAATGGGCCGCTGAAAGGGTGCCCGATAG')
>>> messenger_rna = coding_dna.transcribe()
>>> messenger_rna
Seq('AUGGCCAUUGUAAUGGGCCGCUGAAAGGGUGCCCGAUAG')
```

# Translation

- It can translate RNA or directly from the coding strand DNA sequence

```
>>> from Bio.Seq import Seq
>>> messenger_rna = Seq("AUGGCCAUUGUAAUGGGCCGCUGAAAGGGUGCCCGAUAG")
>>> messenger_rna
Seq('AUGGCCAUUGUAAUGGGCCGCUGAAAGGGUGCCCGAUAG')
>>> messenger_rna.translate()
Seq('MAIVMGR*KGAR*')

>>> from Bio.Seq import Seq
>>> coding_dna = Seq("ATGGCCATTGTAATGGGCCGCTGAAAGGGTGCCCGATAG")
>>> coding_dna
Seq('ATGGCCATTGTAATGGGCCGCTGAAAGGGTGCCCGATAG')
>>> coding_dna.translate()
Seq('MAIVMGR*KGAR*')
```

- Notice that in the protein sequences, in addition to the end stop character, there is an internal stop as well.
- Talk about some optional arguments: including different translation tables (Genetic Codes).

Code with BioPython

# Pairwise alignment in Biopython

- Module Bio.pairwise2
- Many functions all starting with “local”or “global”
- Followed by 2 characters:
  - The first
    - x = match is 1, mismatch is 0
    - m = user can define value for match and mismatch
    - d = user can pass a substitution matrix
  - The second
    - X = no gap penalty
    - S user can define affine gap penalties (opening and extension)

examples

# Homework

- From uniport download the FASTA sequence for the human hemoglobin subunit alpha and mouse hemoglobin subunit alpha and use biopython the perform local and global alignments considering a simple addictive objective function with no gap penalty and an objective function using BLOSUM50 and an affine gap penalty of [-12,-7].
- Observe the 4 different output and the different scores